mode, an irreversible transition that gives it access to the low-level hardware. Nevertheless, it is still a process and able to talk to and rely on the kernel. The only difference that it uses the VMCALL instruction to make a system call. The Dune approach was later used for research in dataplane operating systems such as IX (Belay, 2017) and finally adapted as the foundation for Google's container solution gVisor (Young, 2019).

Speaking of containers, the cloud is seeing a shift from being a platform for tenants to run virtual machines (specified by a virtual disk image) to a platform used by tenants to run containers specified as Dockerfiles and coordinated by orchestrators such as Kubernetes. Often, the container runs within a virtual machine but the guest operating system is now run by the cloud provider. The latest trend, called "serverless", further decouples the application logic from its environment (Shahrad et al., 2020). The idea here is to allow a service to be automatically spun up to serve a single function (an RPC over HTTPS) without managing any aspect of its operating system environment. Amazon calls its serverless technology firecracker (Barr, 2018) and Google calls it gVisor (Young, 2019). Serverless computing has gained even more popularity with the adoption of the Function-as-a-Service (FAAS) model (Kim and Lee, 2019). Of course, the operating system and indeed the server both still exist in serverless operations. But they are hidden from the developer.

The cloud is therefore much more complex than it was in the beginning with the original AWS EC2 model of virtual machines: networks are virtualized, and applications are encapsulated into containers and serverless models that decouple them from the underlying layers. The cloud is more complex, but also more powerful and central to nearly every computing organization. With this importance comes another consideration: trust. Specifically, how much must a tenant trust the cloud service provider ? The correct answer is obviously "as little as necessary."

To achieve this goal of minimal trust dependencies, Intel introduced SGX as the first architectural extension for "confidential computing." SGX creates enclaves, which are isolated in hardware from the host operating system and other applications, with the content of memory and registers cryptographically encrypted by the hardware. In a way, technologies such as SGX are similar to virtualization extensions: they create new ways for software to isolate itself from each other. SGX has notably been used in research to run entire operating systems such as in Haven (Baumann, 2015) or Linux containers such as in SCONE (Arnautov, 2016).

7.13 SUMMARY

Virtualization is the technique of simulating a computer, but with high performance. Typically one computer runs many virtual machines at the same time. This technique is widely used in data centers to provide cloud computing. In this chapter we looked at how virtualization works, especially for paging, I/O, and multicore systems. We also studied an example: VMWare.

PROBLEMS

1. Give a reason why a data center might be interested in virtualization.
2. Give a reason why a company might be interested in running a hypervisor on a machine that has been in use for a while.
3. Give a reason why a software developer might use virtualization on a desktop machine being used for development.
4. Give a reason why an individual at home might be interested in virtualization. Which type of hypervisor would probably be best for a home user?
5. Why do you think virtualization took so long to become popular? After all, the key paper was written in 1974 and IBM mainframes had the necessary hardware and software throughout the 1970s and beyond.
6. What are the three main requirements for designing hypervisors?
7. Name two kinds of instructions that are sensitive in the Popek and Goldberg sense.
8. Name three machine instructions that are not sensitive in the Popek and Goldberg sense.
9. What is the difference between full virtualization and paravirtualization? Which do you think is harder to do? Explain your answer.
10. Does it make sense to paravirtualize an operating system if the source code is available? What if it is not?
11. Consider a type 1 hypervisor that can support up to n virtual machines at the same time. PCs can have a maximum of four disk primary partitions. Can n be larger than 4? If so, where can the data be stored?
12. Briefly explain the concept of process-level virtualization.
13. Why do type 2 hypervisors exist? After all, there is nothing they can do that type 1 hypervisors cannot do and the type 1 hypervisors are generally more efficient as well.
14. Is virtualization of any use to type 2 hypervisors?
15. Why was binary translation invented? Do you think it has much of a future? Explain your answer.
16. Explain how the x86's four protection rings can be used to support virtualization.
17. State one reason as to why a hardware-based approach using VT-enabled CPUs can perform poorly when compared to translation-based software approaches.
18. Give one case where a translated code can be faster than the original code, in a system using binary translation.
19. VMware does binary translation one basic block at a time, then it executes the block and starts translating the next one. Could it translate the entire program in advance and then execute it? If so, what are the advantages and disadvantages of each technique?
20. What is the difference between a pure hypervisor and a pure microkernel?

21. Briefly explain why memory is so difficult to virtualize well in practice? Explain your answer.

22. Running multiple virtual machines on a PC is known to require large amounts of memory. Why? Can you think of any ways to reduce the memory usage? Explain.

23. Explain the concept of shadow page tables, as used in memory virtualization.

24. One way to handle guest operating systems that change their page tables using ordinary (nonprivileged) instructions is to mark the page tables as read only and take a trap when they are modified. How else could the shadow page tables be maintained? Discuss the efficiency of your approach versus the read-only page tables.

25. Why are balloon drivers used? Is this cheating?

26. Describe a situation in which balloon drivers do not work.

27. Explain the concept of deduplication as used in memory virtualization.

28. Computers have had DMA for doing I/O for decades. Did this cause any problems before there were I/O MMUs?

29. What is a virtual appliance? Why is such a thing useful?

30. PCs differ in minor ways at the very lowest level, things like how timers are managed, how interrupts are handled, and some of the details of DMA. Do these differences mean that virtual appliances are not actually going to work well in practice? Explain your answer.

31. Give one advantage of cloud computing over running your programs locally. Give one disadvantage as well.

32. Give an example of IAAS, PAAS, SAAS, and FAAS.

33. Why is virtual machine migration important? Under what circumstances might it be useful?

34. Migrating virtual machines may be easier than migrating processes, but migration can still be difficult. What problems can arise when migrating a virtual machine?

35. Why is migration of virtual machines from one machine to another easier than migrating processes from one machine to another?

36. What is the difference between live migration and the other kind (dead migration)?

37. What were the three main requirements considered while designing VMware?

38. Why was the enormous number of peripheral devices available not a problem when VMware Workstation was first introduced?

39. VMware ESXi has been made very small. Why? After all, servers at data centers usually have tens of gigabytes of RAM. What difference does a few tens of megabytes more or less make?

40. We have seen that hypervisor-based virtual machines provides better isolation than containers. This is clearly an advantage from a security point of view. However, can you think of any security advantages that containers might have over virtual machines?